



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

**Faculty of Engineering and Information Sciences
School of Computing & Information Technology**

CSIT321: Project
Capstone Project - Autumn 2026

Personal Reflection

Student

Chelsea Okan - 8438675

University of Wollongong

Liverpool Campus

Subject Coordinator

Dr. John Le

Dr. Khoa Nguyen

25th May, 2026

1. Overview of My Role

My role in the SmartResearch capstone project expanded significantly beyond a single assigned position. I was originally responsible for project coordination, documentation, quality assurance, and swing development support. As the project progressed, my responsibilities became substantially broader. I maintained the GitHub repository as the project's central workspace, managed meeting records and advisor communication, reviewed the alignment between the implemented system and the written documentation, and ensured that each submission was coherent, complete, and presented to an acceptable standard.

In practice, I became a stabilising contributor within the project. A large part of my role involved taking incomplete, inconsistent, or fragmented work and turning it into material that could actually be submitted. This included writing major report sections, editing and restructuring group contributions, preparing presentation material, creating diagrams, checking technical accuracy, revising scripts, recording missing voice-over content, auditing contribution claims, and assembling the final documentation package.

The project was initially expected to be delivered by a much larger team of approximately six or seven members. By the later stages, however, it had effectively become a two-person working project, with limited intermittent contribution from a third member. This reduction in active capacity shaped the entire delivery process. It affected what could realistically be completed, how work needed to be prioritised, how much time remained for integration and testing, and how often I had to move between project management, technical review, documentation, and submission preparation.

2. Major Work Completed

I contributed across the full project lifecycle rather than within one isolated area. Early in the project, I set up and maintained the GitHub repository, created the initial frontend and backend foundations, connected the frontend and backend, added CORS configuration, and confirmed that the system had a working baseline. This was an important technical starting point because it gave later development work a usable structure rather than leaving the project as a collection of disconnected ideas.

I also contributed to the early backend structure and processing foundations. This included schemas and service modules related to documents, metadata, upload responses, summarisation, clustering, text retrieval, search, PDF extraction, OCR, metadata enrichment, embeddings, and semantic search. Although later implementation work refined and extended these areas, the early foundation established the shape of the system and gave the project a workable technical direction.

Across the written submissions, I wrote, revised, expanded, shortened, integrated, and standardised major sections of the documentation. This included the problem description, project scope, project need, risk management, stakeholder analysis, skills identification, meeting records, references, contribution material, design-progress documentation, requirements traceability, technical explanations, system-alignment content, testing documentation, installation guidance, project-closeout material, and the final technical report and user manual.

A substantial part of this work was not simply writing new content. It involved making sure the written material accurately reflected the implemented system. As the project changed, I repeatedly had to compare documentation against the repository, remove unsupported claims, revise descriptions of incomplete features, and distinguish between what had been planned, what had been partially implemented, and what could honestly be presented as part of the final prototype.

I also completed a significant amount of presentation work. I created and finalised the majority of the presentation structure, slide content, Gantt-chart material, speaker notes, and scripts. Where allocated material was missing or not usable, I revised or completed additional sections so the final presentations could still be delivered properly. During the marketing and technical presentation stage, I also recorded additional voice-over material when required.

For the final submission stage, I prepared the technical report, user manual, requirements-traceability content, testing documentation, contribution records, individual reflection, installation guidance, repository documentation, diagrams, screenshot plan, and final packaging structure. I reviewed terminology, cross-references, feature descriptions, repository evidence, and submission readiness while ensuring that the submitted material reflected the actual state of the prototype rather than overstating incomplete work.

3. Challenges Faced

The largest challenge was the gap between the project's original team structure and the actual working capacity available during delivery. The workload had been planned around a substantially larger group, but the active team became much smaller over time. This issue was identified early and raised with the project advisor from the beginning of the project. Despite repeated reminders, follow-ups, and attempts to clarify responsibilities, the imbalance continued across multiple assessment stages.

The impact became especially clear close to the final submission deadline. Some contributors participated in presentations and recorded prototype material during earlier stages, but major technical work was still outstanding approximately one week before hand-in. No final code had been pushed despite repeated reminders. This was not simply an inconvenience. The final documentation, screenshots, testing evidence, technical verification, and repository updates all depended on a stable merged build.

Late implementation compressed the time available for everything that was supposed to follow it. Integration, testing, debugging, documentation, formatting, and quality assurance all became harder to complete properly because they were forced into the same final window. Accurate technical documentation was particularly difficult when implementation details were missing, unclear, or not provided in a form that could be used directly. In those cases, I had to reconstruct system behaviour from the repository, previous submissions, and the working prototype.

The final submission period made the consequences of the uneven workload particularly clear. On the evening before the deadline, several essential submission tasks were still incomplete, including screenshot evidence, diagram placement, documentation verification, contribution records, and final repository checks. These were not isolated formatting issues. They required repeated comparison between the implemented system, the repository history, the technical report, the user manual, and the contribution table in order to determine what had actually been completed and what could be represented accurately. As a result, the final stage involved not only assembling the submission package, but also reconstructing and verifying project evidence under significant time pressure.

The project itself also involved a significant technical learning curve. I had previously worked on independent web projects, including a hosted personal site, but SmartResearch operated at a different scale. It required frontend work, backend work, API communication, system integration, PDF processing, metadata handling, summarisation, semantic search, clustering, export behaviour, testing, and user-facing documentation. I had not previously worked with React and Vite to this extent, so I had to develop a stronger understanding of routing, component behaviour, interface states, and frontend-backend integration in order to contribute meaningfully.

The most difficult part was not any single task. It was the need to keep moving between technical work, documentation, project coordination, and submission recovery without losing track of how each part affected the others. The project could not be treated as separate pieces. A change in the interface affected the user manual. A missing endpoint affected the technical report. A delayed code push affected testing evidence. A vague contribution claim affected the final records. Everything needed to be checked against everything else.

4. Technical and Project-Management Learning

Technically, this project strengthened my understanding of how a web application is structured, integrated, documented, and handed over. I gained practical experience with frontend and backend setup, API communication, CORS configuration, backend endpoints, local storage, processing services, and the relationship between implementation and documentation.

I also developed a much stronger appreciation for interface development. Before this project, I had not used React and Vite in this way. I had to understand how the frontend was structured, how routing worked, how pages connected to API calls, and how interface states needed to reflect backend behaviour. The frontend could not simply look complete. It needed to communicate what the system was actually doing, handle incomplete or failed actions clearly, and remain understandable to users who had not built it.

One thing I had not fully appreciated before was how important handover quality is. In an independent project, unclear setup decisions or undocumented assumptions mainly affect the original developer. In a group project, they affect everybody who needs to build, test, explain, or maintain the system later. Clear repository structure, setup guidance, implementation notes, meeting records, and testing evidence are therefore part of the technical work rather than optional extras.

This project also made me appreciate how quickly a seemingly manageable system can become complex. SmartResearch sounds relatively contained at a high level: upload papers, extract information, summarise them, cluster them, search them, and export the results. In practice, each of those features creates its own technical and documentation requirements. The difficulty was not only making features exist. It was making sure they connected properly, behaved consistently, could be tested, and could be described accurately without overstating what the prototype could do.

From a project-management perspective, I learned that assigning roles is not enough. Responsibilities need to be tied to concrete outputs, internal deadlines, and evidence of completion. A role title does not reduce project risk unless it results in something usable: working code, documentation, testing evidence, presentation material, or another completed artefact.

My exposure to professional project-delivery practices also changed how I viewed the project. Learning more about how teams manage deliverables, ownership, milestone tracking, escalation, and quality control made the weaknesses in our own process more obvious. It reinforced why formal processes matter. Work does not stay manageable when responsibilities are left to informal follow-up, vague commitments, and good intentions.

5. Reflection on Teamwork and Contribution

The final contribution pattern is visible across the repository history, meeting records, and contribution table. The project was not delivered through an evenly distributed workload. The majority of the sustained implementation, coordination, documentation, integration, and submission-preparation work was carried by two consistently active contributors, with limited additional contribution from a third member.

My own contribution covered project coordination, documentation, quality assurance, technical alignment, presentation writing, user-facing documentation, contribution tracking, repository auditing, diagram preparation, and final packaging. Other contributors completed specific implementation refinements, interface work, marketing assets, presentation delivery, demo recording, and late-stage feature work. However, the overall workload remained uneven across the project lifecycle.

A major part of my contribution involved recovery work. I often had to take fragmented, incomplete, or inconsistent material and turn it into something coherent and submission-ready. This included rewriting sections, restructuring documents, correcting technical descriptions, revising scripts, recording missing voice-over content, checking repository evidence, preparing diagrams, reviewing screenshots, and ensuring that the system was not misrepresented in the final reports.

This recovery work became especially significant during the final submission stage. Tasks that should have been completed progressively, or delegated across the original team, remained unresolved close to the deadline. I had to move between technical verification, documentation editing, diagram preparation, screenshot planning, repository review, contribution auditing, and final packaging while still checking that the submitted material did not overstate the prototype. This made the final delivery process substantially more demanding than a normal documentation and quality-assurance role.

This protected the project outcome, but it also came with a cost. Time spent recovering missing or incomplete work was time that could not be spent on deeper testing, stronger technical refinement, cleaner integration, or more deliberate review. By the final stage, a large part of the process had shifted away from the structured workflow I originally wanted to maintain and toward keeping the project stable enough to submit.

The experience clarified some of my strengths. I was able to identify gaps, impose structure, maintain written records, keep work moving, align technical and written material, and produce submission-ready documentation under pressure. It also clarified what I would approach differently in the future. Taking on extra work can protect a project, but it can also conceal the extent of an imbalance unless the work is recorded honestly and responsibilities are escalated early.

6. Future Improvements

Future improvements should focus on both the system and the project-delivery process.

Technically, SmartResearch would benefit from stronger batch-upload validation, clearer malformed-PDF handling, more consistent summary formatting, improved cluster labelling, stronger export support, and more complete end-to-end testing. The final prototype demonstrates a meaningful workflow, but several features would need further refinement before the system could be treated as a mature product rather than a capstone prototype.

The documentation should also be maintained alongside the code. When system behaviour changes, the technical report, user manual, testing records, and marketing material should be updated at the same time rather than reconstructed at the end. This would reduce inconsistencies and make final verification significantly easier.

From a project-management perspective, future projects should include a clearer process for responding when the active group size changes. If a project begins with a larger team but later becomes dependent on fewer contributors, the scope, workload, and responsibilities should be reviewed formally rather than absorbed informally by the remaining active members.

Task tracking should also be stricter. Each task should have a named owner, a due date, a specific output, and evidence of completion. For technical work, this could include commits, implementation notes, testing records, or working artefacts. For documentation or presentation work, it should mean usable draft content rather than rough fragments or incomplete placeholders.

I would also set internal deadlines well before official submission dates. This would leave enough time for review, correction, integration, testing, formatting, and final quality assurance. If a task was not delivered by its internal deadline, it should be escalated and formally reallocated rather than absorbed close to submission.

Overall, SmartResearch was brought to a submission-ready state through sustained technical work, documentation, integration, and quality control. The process required far more correction and recovery work than it should have, but it also made the core lesson of the project very clear. Successful delivery depends not only on technical ability, but also on having a sufficiently active team, clear ownership, reliable contribution, accurate documentation, and honest records of what was actually completed.

7. Peer Assessment

Noor Ahmed

Noor was a consistently active contributor during the later stages of the project. He completed backend refinement, testing, debugging, technical-feature documentation support, testing-evidence documentation, visual and marketing materials, and post-snapshot runtime verification of the final build. His work was usable, responsive to project needs, and important to the successful completion of the final submission package.

Noor also provided support during the final consolidation period by checking the updated implementation, confirming that the final build was running successfully, and contributing material that could be integrated directly into the technical report and testing documentation. His contribution was substantial and reliable.

Nishad Gyawali

Nishad contributed presentation delivery, prototype-demo recording, technical-video narration, and a substantial late implementation update during the morning of submission. The final implementation update included Export-page and PDF-report refinements, clustering refinements, AI-summary and recovery integration, recommendation updates, and supporting prototype-integration work. This update was runtime checked by Noor and recorded in the contribution table as a post-snapshot contribution.

However, the timing of this implementation work significantly limited the opportunity for broader regression testing, screenshot capture, documentation alignment, and integration review before submission. For most of the project lifecycle and the earlier final-consolidation period, Nishad's contribution remained limited and intermittent. Although the late implementation update should be recognised, it did not remove the impact of the earlier delays on the workload carried by the other active contributors.